LAB # 9 **SIGNALS**

Objectives:

- ***Demonstrate the use of signal handling system calls in user-level programs.***
- ***Analyze the behavior of processes under different signal events to understand asynchronous communication.***
- ***Implement custom signal handlers in C to manage process-level signal responses efficiently.***

Pre-Lab Theory:

SIGNALS:

A signal is a software notification to a process of an event. A signal is generated when the event that causes the signal occurs. A signal is delivered when the process takes action based on that signal. The lifetime of a signal is the interval between its generation and its delivery. A signal that has been generated but not yet delivered is said to be pending. There may be considerable time between signal generation and signal delivery. The process must be running on a processor at the time of signal delivery.

A process catches a signal if it executes a signal handler when the signal is delivered. A program installs a signal handler by calling sigaction with the name of a user-written function. The sigaction function may also be called with SIG_DFL or SIG_IGN instead of a handler. The SIG_DFL means take the default action, and SIG_IGN means ignore the signal. Neither of these actions is considered to be "catching" the signal. If the process is set to ignore a signal, that signal is thrown away when delivered and has no effect on the process.

The action taken when a signal is generated depends on the current signal handler for that signal and on the process signal mask. The signal mask contains a list of currently blocked signals. It is easy to confuse blocking a signal with ignoring a signal. Blocked signals are not thrown away as ignored signals are. If a pending signal is blocked, it is delivered when the process unblocks that signal. A program blocks a signal by changing its process signal mask, using sigprocmask. A program ignores a signal by setting the signal handler to SIG_IGN, using sigaction.

SYNOPSIS

```
#include <signal.h>

int kill(pid_t pid, int sig);

int raise(int sig);

unsigned alarm(unsigned seconds);
```

SIGNAL Installation:

```
int sigaction(int sig, const struct sigaction *restrict act, struct sigaction *restrict oact);
```

The sigaction function allows the caller to examine or specify the action associated with a specific signal. The sig parameter of sigaction specifies the signal number for the action.

The act parameter is a pointer to a struct sigaction structure that specifies the action to be taken.

The oact parameter is a pointer to a struct sigaction structure that receives the previous action associated with the signal.

If act is NULL, the call to sigaction does not change the action associated with the signal.

If oact is NULL, the call to sigaction does not return the previous action associated with the signal.

SIGNAL MASKING:

*int sigprocmask(int how, const sigset_t *restrict set, sigset_t *restrict oset);*

A process can temporarily prevent a signal from being delivered by blocking it. Blocked signals do not affect the behavior of the process until they are delivered. The process signal mask gives the set of signals that are currently blocked. The signal mask is of type sigset_t.

Blocking a signal is different from ignoring a signal. When a process blocks a signal, the operating system does not deliver the signal until the process unblocks the signal. A process blocks a signal by modifying its signal mask with sigprocmask. When a process ignores a signal, the signal is delivered and the process handles it by throwing it away. The process sets a signal to be ignored by calling sigaction with a handler of SIG_IGN,

Specify operations (such as blocking or unblocking) on groups of signals by using signal sets of type sigset_t. Signal sets are manipulated by the five functions listed in the following synopsis box. The first parameter for each function is a pointer to a sigset_t. The sigaddset adds signo to the signal set, and the sigdelset removes signo from the signal set. The sigemptyset function initializes a sigset_t to contain no signals; sigfillset initializes a sigset_t to contain all signals. Initialize a signal set by calling either sigemptyset or sigfillset before using it. The sigismember reports whether signo is in a sigset_t.

SYNOPSIS

```
int sigaddset(sigset_t *set, int signo);

int sigdelset(sigset_t *set, int signo);

int sigemptyset(sigset_t *set);

int sigfillset(sigset_t *set);

int sigismember(const sigset_t *set, int signo);
```

The how parameter, which specifies the manner in which the signal mask is to be modified, can take on one of the following three values.

SIG_BLOCK: add a collection of signals to those currently blocked

SIG_UNBLOCK: delete a collection of signals from those currently blocked

SIG_SETMASK: set the collection of signals being blocked to the specified set

Task 1:

Display all the signal name to the standard output using kill command

Command:

Output:

Task 2:

Open two terminal windows and run an executable file in one terminal to make it a process. Check the process id of that process from the other terminal and send SIGKILL to that process.

Ouiput of (ps command):   *Command to send the SIGKILL to the process:*

Observation: Process is terminated or not (yes/no)

Task 3:

Write the code to install SIGINT with its handler function using sigaction installer. The handler function should display the message "SIGINT IS CAUGHT". The handler function will be out of main() function but the signal association with signal handler will be in the main function.

Task 4:

Modify the code in task 3 add the code with infinite loop in it. Compile and run the program and press Ctrl-C to generate SIGINT signal and observe the output.

Output:

Task 5:

Generate any 10 signals from the signals list displayed in Task 1 through raise() call. Associate these signals to one signal handler which is responsible to catch those signals and display the name of signal which is caught.

Output:

Task 6:

Modify the Task 3 code to Block/Mask the signal SIGINT and verify that the signal handler will not display the message "SIGINT IS CAUGHT" when you press Ctrl-C

Code

Observation: whether the message "SIGINT IS CAUGHT" displayed or not

Task 7:

Modify the Task 6, after blocking code add sleep(10) instruction and unblock the SIGINT signal. Compile and Run the program. Verify that first 10 seconds Pressing Ctrl-C will not have any effect. Write the output of the code.

Output: How many times the message "SIGINT is CAUGHT" displayed and why?